

# Java Programming Fundamentals

Comprehensive Core Concepts, Code Examples & Practice Assessments

## 1. String Concatenation & Multi-Line Literals

In Java, the plus sign ( `+` ) performs dual roles depending on its operands. When used with numeric values, it acts as an addition operator. When used with strings, it operates as the **string concatenation operator**, joining two string representations into a single consolidated string.

If a string is concatenated with a non-string value (such as a primitive number or boolean), Java automatically converts the primitive operand into its string representation before concatenating.

### Caution: String Literals Across Lines

A literal string in Java cannot directly span multiple physical lines in the source code. The following statement results in a compile-time error:

```
// COMPILER ERROR: String literal cannot span multiple lines directly  
System.out.println("Introduction to Java Programming,  
    by Y. Daniel Liang");
```

**Solution:** To split a long string across multiple lines in your source file for readability, break the string into smaller string substrings and use the concatenation operator ( `+` ) to combine them:

```
System.out.println("Introduction to Java Programming, " +  
    "by Y. Daniel Liang");
```

## Topic Assessment: String Concatenation

**EASY** 1. What is the output of `System.out.println("Java " + 8 + 2);` ?

- A) Java 10
- B) Java 82
- C) Java 8 2
- D) Compilation Error

Answer: B) Java 82 (Evaluation goes left to right: "Java " + 8 becomes "Java 8", then "Java 8" + 2 becomes "Java 82")

**EASY** 2. What happens if a string literal in Java spans across two lines without quotes on each line?

- A) The compiler automatically inserts a newline character.
- B) It results in a compile-time syntax error.
- C) The program executes and ignores the new line.
- D) A runtime exception is thrown.

Answer: B) It results in a compile-time syntax error.

**MEDIUM** 3. What will be printed by `System.out.println(5 + 5 + " is ten");` ?

- A) 55 is ten
- B) 10 is ten
- C) 5 + 5 is ten
- D) Compilation Error

Answer: B) 10 is ten (Numeric addition  $5 + 5 = 10$  occurs first before string concatenation)

**MEDIUM** 4. Which statement correctly breaks a long string across two code lines?

- A) `System.out.println("Hello \ World");`
- B) `System.out.println("Hello " + "World");`
- C) `System.out.println("Hello" + "World");`
- D) `System.out.println("Hello" & "World");`

Answer: B) `System.out.println("Hello " + "World");`

**HARD** 5. What is the output of `System.out.println("Result: " + (10 + 20));` versus `System.out.println("Result: " + 10 + 20);` ?

- A) Result: 30 and Result: 30
- B) Result: 1020 and Result: 1020
- C) Result: 30 and Result: 1020
- D) Result: 1020 and Result: 30

Answer: C) Result: 30 and Result: 1020 (Parentheses override precedence, evaluating arithmetic first)

## 2. Console Input Using the Scanner Class

Java uses standard stream objects for I/O operations:

- `System.out` refers to the standard output device (by default, the monitor display).
- `System.in` refers to the standard input device (by default, the keyboard).

To accept interactive input from the console, Java provides the `Scanner` class within the `java.util` package. To use it, you create a `Scanner` object bound to `System.in`:

```
Scanner input = new Scanner(System.in);
```

### Common Scanner Methods for Reading Data Types

| Method                    | Description / Return Type                                              |
|---------------------------|------------------------------------------------------------------------|
| <code>nextByte()</code>   | Reads an integer value of type <code>byte</code> (-128 to 127).        |
| <code>nextShort()</code>  | Reads an integer value of type <code>short</code> (-32,768 to 32,767). |
| <code>nextInt()</code>    | Reads an integer value of type <code>int</code> .                      |
| <code>nextLong()</code>   | Reads an integer value of type <code>long</code> .                     |
| <code>nextFloat()</code>  | Reads a floating-point number of type <code>float</code> .             |
| <code>nextDouble()</code> | Reads a floating-point number of type <code>double</code> .            |
| <code>next()</code>       | Reads a single token (string ending before whitespace).                |
| <code>nextLine()</code>   | Reads an entire line of text up to the Enter key press.                |

### Example Program: Computing Circle Area with User Input

```
import java.util.Scanner; // Import Scanner class

public class ComputeAreaWithConsoleInput {
    public static void main(String[] args) {
        // Create a Scanner object
        Scanner input = new Scanner(System.in);

        // Prompt the user to enter a radius
        System.out.print("Enter a number for radius: ");
        double radius = input.nextDouble();

        // Compute area
        double area = radius * radius * 3.14159;

        // Display results
        System.out.println("The area for the circle of radius " +
                           radius + " is " + area);
    }
}
```

## Topic Assessment: Console Input & Scanner

**EASY** 1. Which package contains the `Scanner` class?

- A) `java.lang`
- B) `java.io`
- C) `java.util`
- D) `java.system`

Answer: C) `java.util`

**EASY** 2. What object represents the standard keyboard input stream in Java?

- A) `System.out`
- B) `System.in`
- C) `System.err`
- D) `Console.input`

Answer: B) `System.in`

**MEDIUM** 3. Which `Scanner` method reads a full sentence including spaces until the Enter key is pressed?

- A) `next()`
- B) `readString()`
- C) `nextLine()`
- D) `getline()`

Answer: C) `nextLine()` (Note: `next()` stops at whitespace)

**MEDIUM** 4. If a user enters "3.14159" into console input, which `Scanner` call will fail with an exception?

- A) `input.nextDouble()`
- B) `input.nextFloat()`
- C) `input.nextInt()`
- D) `input.next()`

Answer: C) `input.nextInt()` (Throws `InputMismatchException` because 3.14159 is floating-point)

**HARD** 5. What issue occurs if you call `input.nextInt()` followed immediately by `input.nextLine()`?

- A) A compile error occurs due to incompatible types.
- B) `nextLine()` consumes the leftover newline character ( ) from pressing Enter, returning an empty string.
- C) The program blocks permanently waiting for input.
- D) The `Scanner` instance is automatically closed.

Answer: B) `nextLine()` consumes the trailing newline token left behind by `nextInt()`.

### 3. Identifiers in Java

---

Identifiers are user-defined names given to program elements such as variables, constants, methods, classes, and packages.

#### Rules for Valid Java Identifiers:

- An identifier is a sequence of characters consisting of letters, digits, underscores ( `_` ), and dollar signs ( `$` ).
- An identifier **must start** with a letter, an underscore ( `_` ), or a dollar sign ( `$` ). It **cannot start with a digit**.
- An identifier **cannot be a Java reserved keyword** (e.g., `class` , `public` , `int` ).
- An identifier cannot be the literal constants `true` , `false` , or `null` .
- Identifiers can be of any length.

#### Important Characteristics:

- **Case Sensitivity:** Java is strictly case-sensitive. Thus, `area` , `Area` , and `AREA` are three completely distinct identifiers.
- **Dollar Sign Convention:** Although `$` is syntactically legal, by standard convention, the `$` character should be avoided and is reserved for mechanically generated code.

## Topic Assessment: Java Identifiers

**EASY** 1. Which of the following is a valid Java identifier?

- A) 2ndVariable
- B) my-variable
- C) \_score\$
- D) class

Answer: C) \_score\$ (Cannot start with digit [A], hyphens illegal [B], reserved keyword [D])

**EASY** 2. How does Java treat identifiers `totalValue` and `TotalValue` ?

- A) As identical variables
- B) As two completely distinct identifiers
- C) Triggers a compiler warning
- D) Causes a runtime collision

Answer: B) As two completely distinct identifiers (Java is case-sensitive)

**MEDIUM** 3. Identify the invalid Java identifier from the following list: `miles` , `$4` , `apps` , `4#R` , `radius` .

- A) \$4
- B) 4#R
- C) miles
- D) apps

Answer: B) 4#R (Starts with a digit '4' and contains special character '#')

**MEDIUM** 4. Which set of words are all forbidden as variable names in Java?

- A) `public` , `static` , `void`
- B) `Main` , `System` , `String`
- C) `integer` , `doubleValue` , `counter`
- D) `input` , `output` , `scan`

Answer: A) `public` , `static` , `void` (All are reserved keywords in Java)

**HARD** 5. Is `true` a valid Java identifier? Why or why not?

- A) Yes, because it starts with a lowercase letter.
- B) No, because it is a boolean literal constant in Java.
- C) Yes, provided it is assigned an integer value.
- D) No, because it is shorter than 5 characters.

Answer: B) No, reserved boolean literals (`true`, `false`, `null`) cannot be used as identifiers.

## 4. Multiple Variable Assignments & Type Compatibility

Java allows chained assignment statements to assign a common value to multiple variables simultaneously:

```
i = j = k = 1;
```

This single statement is evaluated right-to-left and is mathematically and logically equivalent to:

```
k = 1;  
j = k;  
i = j;
```

#### Type Compatibility Rule:

In an assignment statement, the data type of the variable on the left side of `=` must be compatible with the data type of the value/expression on the right side.

For example: `int x = 1.0;` is **illegal** because `1.0` is a `double` (64-bit floating point), which cannot fit implicitly into an `int` (32-bit integer) without explicit type casting.

## Topic Assessment: Assignments & Compatibility

**EASY** 1. What is the value of `a` after executing `int a, b, c; a = b = c = 50; ?`

- A) 0
- B) 50
- C) Compilation Error
- D) Undefined

Answer: B) 50

**EASY** 2. Why does `int x = 5.5;` cause a compiler error?

- A) Variable `x` must be capitalized.
- B) Cannot assign a double floating-point literal to an `int` variable without explicit casting.
- C) Semi-colon is placed incorrectly.
- D) Numbers with decimals are not permitted in Java.

Answer: B) Primitive narrowing assignment requires explicit type casting.

**MEDIUM** 3. Given `double x = 10;`, will this statement compile successfully?

- A) No, because 10 is an integer and `x` is double.
- B) Yes, implicit widening conversion converts integer 10 to double 10.0.
- C) No, requires explicit casting like `(double)10`.
- D) Compiles but causes runtime error.

Answer: B) Yes, widening from `int` to `double` occurs automatically.

**MEDIUM** 4. In the chained assignment `x = y = z = 100;`, in what order are assignments executed?

- A) Left to Right
- B) Right to Left
- C) Simultaneous
- D) Arbitrary based on compiler optimization

Answer: B) Right to Left (Assignment operator '=' has right-to-left associativity).

**HARD** 5. What happens when compiling: `int a; double b = 4.5; a = b; ?`

- A) `a` becomes 4.
- B) `a` becomes 4.5.
- C) Compile error: "possible loss of precision".
- D) Runtime Exception.

Answer: C) Compile error: "possible loss of precision" (Requires explicit cast `a = (int)b;`).

## 5. Named Constants (The `final` Keyword)

A **named constant** (or simply *constant*) is an identifier representing a permanent value that cannot be modified during the execution of a program.

## Syntax for Declaring Constants:

```
final datatype CONSTANTNAME = value;
```

### Rule: Initialization

A constant **must be declared and initialized in the same statement** or prior to any read operation within constructors/initializers.

## Benefits of Using Constants:

1. **Avoid Repeated Hardcoding:** Eliminates repeating magic numbers (e.g., typing `3.14159` across multiple formulas).
2. **Single Source of Change:** If the value needs updating (e.g., updating tax rate from `0.05` to `0.06`), it only needs to be modified in one location.
3. **Code Readability:** Meaningful identifier names (e.g., `PI` or `MAX_USERS`) make code self-documenting.

```
public class ComputeArea {
    public static void main(String[] args) {
        final double PI = 3.14159; // Declare constant

        double radius = 20;
        double area = radius * radius * PI;

        System.out.println("Area: " + area);
    }
}
```

## Topic Assessment: Named Constants

**EASY** 1. Which Java keyword is used to declare a constant?

- A) `const`
- B) `static`
- C) `final`
- D) `constant`

Answer: C) `final`

**EASY** 2. What happens if you try to modify the value of a `final` variable after initialization?

- A) Value is modified silently.
- B) Compilation error occurs.
- C) Runtime exception is thrown.
- D) Value reverts back at program termination.

Answer: B) Compilation error occurs ("cannot assign a value to final variable").

**MEDIUM** 3. According to Java naming conventions, how should constants be styled?

- A) camelCase
- B) PascalCase
- C) ALL\_CAPS\_WITH\_UNDERSCORES
- D) lowercase\_with\_underscores

Answer: C) ALL\_CAPS\_WITH\_UNDERSCORES (e.g., `MAX_VALUE`, `PI`)

**MEDIUM** 4. Which statement correctly declares a constant floating-point value for gravity?

- A) `constant double GRAVITY = 9.8;`
- B) `final double GRAVITY = 9.8;`
- C) `double final GRAVITY = 9.8;`
- D) `final GRAVITY = 9.8;`

Answer: B) `final double GRAVITY = 9.8;`

**HARD** 5. Which of the following is NOT a direct benefit of using named constants?

- A) Reduces code redundancy.
- B) Increases program execution speed by converting code to bytecode faster.
- C) Improves code maintainability when values change.
- D) Enhances code readability by replacing arbitrary numbers with descriptive names.

Answer: B) Constants enhance readability and maintainability, not compiler velocity.

## 6. Java Naming Conventions

Adhering to standard Java naming conventions ensures consistency, readability, and maintainability across corporate software teams.

| Element Type                    | Naming Convention                                                                                      | Examples                                                                                                |
|---------------------------------|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>Variables &amp; Methods</b>  | Use <b>lowercase</b> . For multi-word names, use <b>camelCase</b> (capitalize starting from 2nd word). | <code>radius</code> , <code>area</code> , <code>computeArea()</code> , <code>showMessageDialog()</code> |
| <b>Classes &amp; Interfaces</b> | Capitalize the first letter of <b>every</b> word ( <b>PascalCase</b> ).                                | <code>ComputeArea</code> , <code>System</code> , <code>JOptionPane</code> , <code>Scanner</code>        |
| <b>Constants</b>                | Capitalize <b>every letter</b> ; separate words with underscores.                                      | <code>PI</code> , <code>MAX_VALUE</code> , <code>MAX_ELEMENTS</code>                                    |

#### Best Practice Tip:

Choose descriptive, complete words rather than cryptic abbreviations. For example, use `numberOfStudents` rather than `numStuds` or `nStud`.

## Topic Assessment: Naming Conventions

**EASY** 1. Which of the following follows standard Java class naming conventions?

- A) `studentRecord`
- B) `student_record`
- C) `StudentRecord`
- D) `STUDENT_RECORD`

Answer: C) `StudentRecord` (PascalCase)

**EASY** 2. What casing style is recommended for Java variable names?

- A) `camelCase`
- B) `UPPERCASE`
- C) `kebab-case`
- D) `snake_case`

Answer: A) `camelCase` (e.g., `totalAccountBalance`)

**MEDIUM** 3. Given the identifier `calculateTaxAmount`, what type of program element does it most likely represent?

- A) Class
- B) Method or Variable
- C) Constant
- D) Package

Answer: B) Method or Variable (camelCase starting with lowercase)

**MEDIUM** 4. Which identifier adheres strictly to Java constant naming conventions?

- A) `interestRate`
- B) `Interest_Rate`
- C) `INTEREST_RATE`
- D) `interest_rate`

Answer: C) `INTEREST_RATE`

**HARD** 5. Which statement regarding Java naming conventions is FALSE?

- A) Naming conventions are enforced strictly by the compiler; violating them causes compilation error.
- B) Classes use PascalCase where every word begins with a capital letter.
- C) Descriptive variable names improve software maintainability.
- D) Method names should typically start with a verb in camelCase.

Answer: A) Naming conventions are human guidelines, not compiler-enforced rules.

## 7. Primitive Numeric Data Types & IEEE 754 Standard

Java provides eight primitive data types, six of which are numeric types: four integer types and two floating-point types.

## Numeric Primitive Data Types Summary

| Type                | Storage Size    | Range                                                         | Description                     |
|---------------------|-----------------|---------------------------------------------------------------|---------------------------------|
| <code>byte</code>   | 8-bit signed    | -128 to 127 ( $-2^7$ to $2^7-1$ )                             | Small integer values            |
| <code>short</code>  | 16-bit signed   | -32,768 to 32,767 ( $-2^{15}$ to $2^{15}-1$ )                 | Medium-small integers           |
| <code>int</code>    | 32-bit signed   | -2,147,483,648 to 2,147,483,647 ( $-2^{31}$ to $2^{31}-1$ )   | Standard integer choice         |
| <code>long</code>   | 64-bit signed   | $-2^{63}$ to $2^{63}-1$ ( $\sim \pm 9.22 \times 10^{18}$ )    | Large integer range             |
| <code>float</code>  | 32-bit IEEE 754 | $\pm 1.4\text{E-}45$ to $\pm 3.4028235\text{E}+38$            | Single precision floating-point |
| <code>double</code> | 64-bit IEEE 754 | $\pm 4.9\text{E-}324$ to $\pm 1.7976931348623157\text{E}+308$ | Double precision floating-point |

### IEEE 754 Floating-Point Standard:

Java conforms to the **IEEE 754** standard approved by the Institute of Electrical and Electronics Engineers for floating-point representation.

`double` (64-bit) provides 14–15 significant decimal digits of precision, making it twice as precise as `float` (32-bit, ~6-7 digits). By default, `double` is preferred for real numbers in Java.

## Topic Assessment: Numeric Data Types

**EASY** 1. What is the storage size of an `int` primitive variable in Java?

- A) 8 bits
- B) 16 bits
- C) 32 bits
- D) 64 bits

Answer: C) 32 bits

**EASY** 2. What is the numerical range for a `byte` data type?

- A) 0 to 255
- B) -128 to 127
- C) -32,768 to 32,767
- D)  $-2^{31}$  to  $2^{31}-1$

Answer: B) -128 to 127

**MEDIUM** 3. Which standard does Java implement for float and double arithmetic?

- A) ISO 9001
- B) IEEE 754
- C) ASCII / UTF-8
- D) ANSI C89

Answer: B) IEEE 754 standard

**MEDIUM** 4. Why is `double` called "double precision"?

- A) It uses double the memory of a long integer.
- B) It provides double the precision (64-bit) compared to float (32-bit).
- C) It automatically doubles any numeric value assigned to it.
- D) It stores two floating-point numbers in a single variable.

Answer: B) It uses 64 bits instead of 32 bits, doubling floating-point precision.

**HARD** 5. Which data type consumes the LEAST amount of memory?

- A) `short`
- B) `byte`
- C) `float`
- D) `int`

Answer: B) `byte` (8 bits / 1 byte)

## 8. Integer Arithmetic & Remainder Operator (%)

Java provides basic arithmetic operators: addition (+), subtraction (-), multiplication (\*), division (/), and remainder/modulo (%).

## Integer Division vs Modulo:

- **Integer Division ( / )**: When both operands are integers, fractional parts are truncated (dropped). E.g., `5 / 2` evaluates to `2`.
- **Remainder / Modulo Operator ( % )**: Returns the integer remainder after division. E.g., `5 % 2` evaluates to `1`.

## Evaluating Time Breakdown Example Program:

```
import java.util.Scanner;

public class DisplayTime {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter an integer for seconds: ");
        int seconds = input.nextInt();

        int minutes = seconds / 60;           // Find minutes
        int remainingSeconds = seconds % 60; // Seconds remaining

        System.out.println(seconds + " seconds is " + minutes +
                           " minutes and " + remainingSeconds + " seconds");
    }
}
```

## Evaluating Negative Operands in Modulo:

In Java, the sign of the result of a remainder operation is **always equal to the sign of the left operand (dividend)**:

- `56 % 6 = 2`
- `78 % -4 = 2`
- `-34 % 5 = -4`
- `-34 % -5 = -4`

## Topic Assessment: Integer Arithmetic & Modulo

**EASY** 1. What is the result of `25 / 4` in Java integer arithmetic?

- A) 6.25
- B) 6
- C) 6.0
- D) 1

Answer: B) 6 (Integer division truncates fractional parts)

**EASY** 2. What is the result of `25 % 4` ?

- A) 6
- B) 1
- C) 0.25
- D) 4

Answer: B) 1 (25 divided by 4 leaves remainder 1)

**MEDIUM** 3. What will `-34 % 5` evaluate to in Java?

- A) 4
- B) -4
- C) 1
- D) -1

Answer: B) -4 (The result inherits the sign of the left operand, -34)

**MEDIUM** 4. How can you rewrite `25 / 4` so that the result is a floating-point number `6.25` ?

- A) `(int)(25 / 4)`
- B) `25 / 4.0`
- C) `25 % 4.0`
- D) `float(25 / 4)`

Answer: B) `25 / 4.0` (Making one operand floating-point promotes division to double)

**HARD** 5. If today is Tuesday (day 2 of week 0..6), what day of the week will it be in 100 days?

- A) Thursday (4)
- B) Friday (5)
- C) Wednesday (3)
- D) Saturday (6)

Answer: A) Thursday  $((2 + 100) \% 7 = 102 \% 7 = 4$ , where 0=Sun, 1=Mon, 2=Tue, 3=Wed, 4=Thu)

## 9. Exponent Operations using `Math.pow()`

Java does not have an exponentiation operator (like `**` or `^` in other languages). To compute exponents  $a^b$ , Java provides the `Math.pow(a, b)` method in the standard `java.lang.Math` class.

## Method Signature & Behavior:

```
Math.pow(double a, double b)
```

- Returns the value of  $a^b$  as a **double**.
- **a** and **b** are parameters representing base and exponent respectively.

## Code Examples:

```
System.out.println(Math.pow(2, 3));    // Displays 8.0  (2^3)
System.out.println(Math.pow(4, 0.5));  // Displays 2.0  (sqrt of 4)
System.out.println(Math.pow(2.5, 2));  // Displays 6.25 (2.5^2)
System.out.println(Math.pow(2.5, -2)); // Displays 0.16 (2.5^-2)
```

## Topic Assessment: Exponent Operations

**EASY** 1. What is the return data type of `Math.pow(2, 3)` ?

- A) `int`
- B) `float`
- C) `double`
- D) `long`

Answer: C) `double` (Always returns double)

**EASY** 2. What is the output of `System.out.println(Math.pow(3, 2));` ?

- A) 9
- B) 9.0
- C) 6.0
- D) 6

Answer: B) 9.0

**MEDIUM** 3. Which statement correctly computes  $2^{3.5}$  in Java?

- A) `Math.pow(3.5, 2)`
- B) `2 ^ 3.5`
- C) `Math.pow(2, 3.5)`
- D) `2 ** 3.5`

Answer: C) `Math.pow(2, 3.5)`

**MEDIUM** 4. Given integers `m` and `r`, which Java expression calculates  $m \cdot r^2$  as a double floating-point result?

- A) `m * r * r`
- B) `m * Math.pow(r, 2)`
- C) `Math.pow(m * r, 2)`
- D) `(m * r) ^ 2`

Answer: B) `m * Math.pow(r, 2)`

**HARD** 5. What is the result of `int result = Math.pow(2, 3);` ?

- A) `result` becomes 8
- B) `result` becomes 8.0
- C) Compile error: "Incompatible types: possible loss of precision from double to int"
- D) Runtime exception

Answer: C) Compile error (Requires explicit cast: `int result = (int)Math.pow(2, 3);`)